

Relational Data, Primary Keys, and Foreign Keys

Python 101 • Day 6 • reason about connected tables before joining

Today's move

from one big flat file



to related tables

keys • references •
relationships

BLOCK 46

What this block covers

30 minutes instruction + 30 minutes guided practice

Split tables

Why repeated reference information often belongs in separate tables.

Identify keys

Primary keys identify rows; foreign keys point to related rows.

Name relationships

One-to-one, one-to-many, and many-to-many awareness.

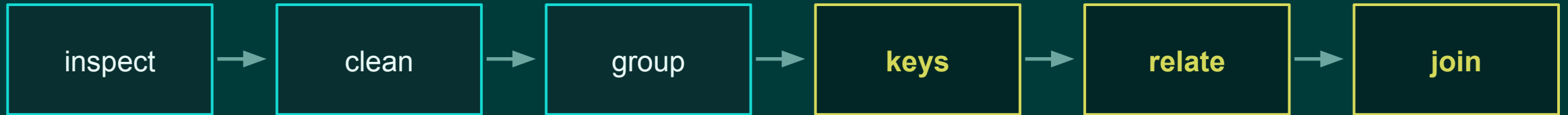
Validate before joins

Check duplicate keys, missing keys, and unmatched references.

Guiding question: What does this value identify, and what does it point to?

Where we are in Day 6

After grouping one table, we now reason across multiple related tables.



This block prepares the thinking needed before pandas merge().

Why not put everything in one CSV?

Repeated descriptive values make the file larger and harder to maintain.

id	source	source_name	org	region	manager
EQ-1	MAINT	Maintenance	Ops	East	Rios
EQ-2	MAINT	Maintenance	Ops	East	Rios
EQ-3	MAINT	Maintenance	Ops	East	Rios
EQ-4	LOG	Logistics	Supply	West	Ng

Problem

Same source details repeat again and again.

Duplication creates more places for typos and inconsistent updates.

A relational structure splits responsibilities

Detail rows store facts; reference tables store shared descriptions.

equipment_records.csv

record_...	source_code	status	
errors			
EQ-1	MAINT	active	0
EQ-2	MAINT	active	2
EQ-4	LOG	inactive	0

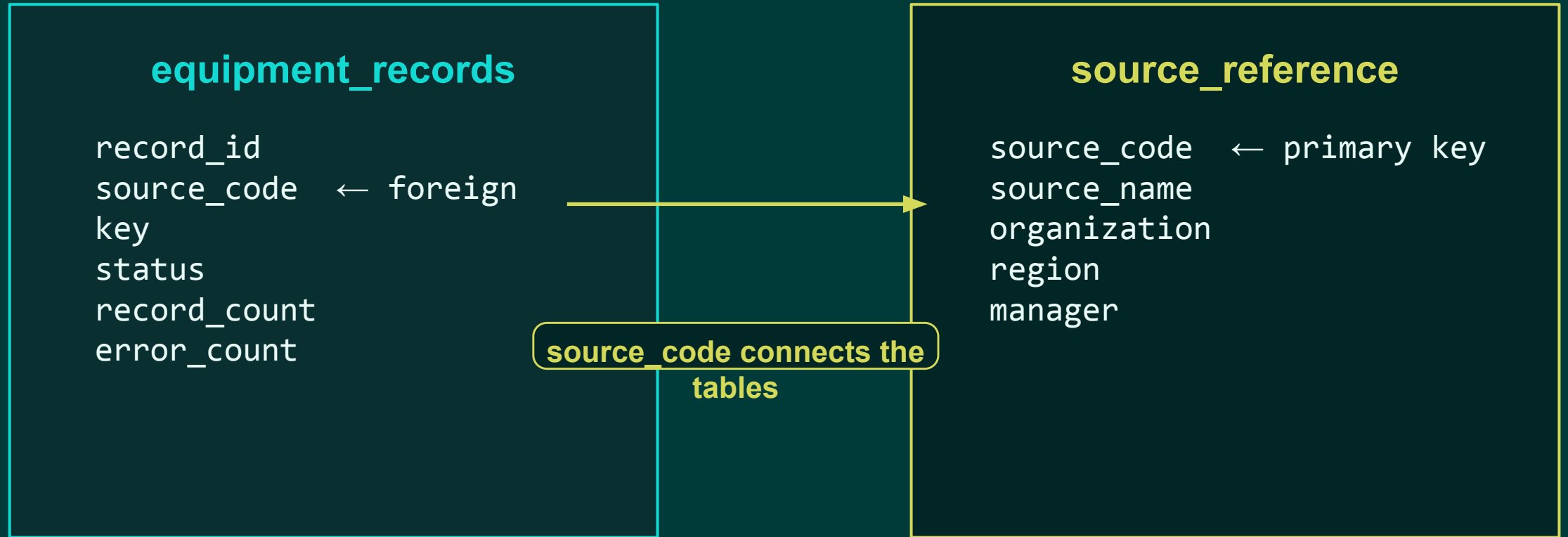
source_reference.csv

source_co...	source_name	org	
region			
MAINT East	Maintenance	Ops	
LOG West	Logistics	Supply	
FIN	Finance	Finance	HQ

The connecting column is **source_code**.

Table relationship mental model

A value in the detail table points to one row in the reference table.



Primary key: one row, one identifier

A primary key uniquely identifies a row within its own table.

source_reference.source_code

source_code	source_name
MAINT	Maintenance
LOG	Logistics
FIN	Finance

Beginner rule

A primary key should identify one and only one row.

Present • unique • stable enough to identify the record

Check key uniqueness in pandas

Before joining, test whether a supposed key is actually unique.

```
print(  
    source_reference["source_code"]  
        .is_unique  
)
```

```
duplicate_sources = source_reference[  
    source_reference.duplicated(  
        subset=["source_code"],  
        keep=False  
    )  
]
```

Interpretation

True means each code appears once.

False means inspect duplicates
before using that key in a relationship.

BLOCK 46

Duplicate keys can multiply rows

If a lookup key appears twice, one detail row may match twice.

id	source		source	manager
errors				
EQ-1	MAINT	0	MAINT	Rios
EQ-2	LOG	1	MAINT	Patel
			LOG	Ng

Unexpected result

EQ-1 may become two output rows because MAINT matched two reference rows.

Know the expected relationship before joining.

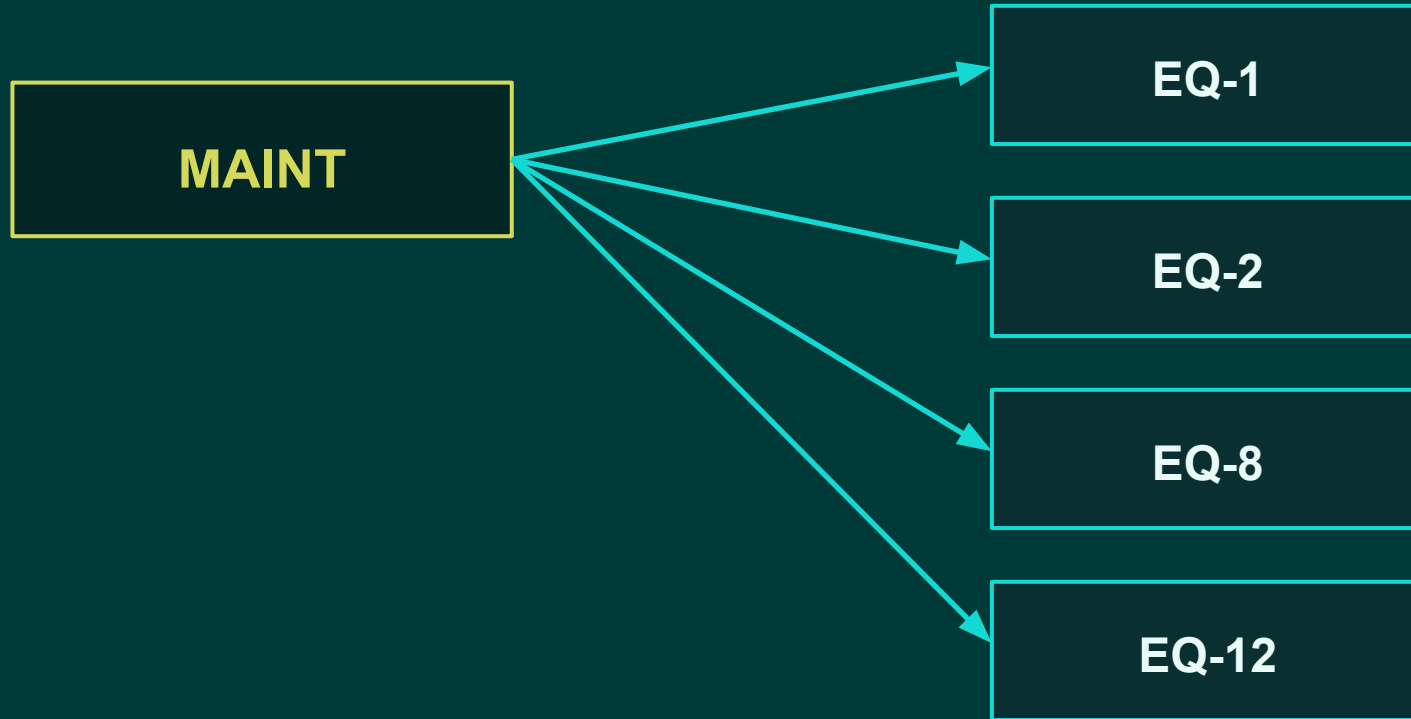
Foreign key: a pointer to another table

A foreign key value says which related row should be looked up.



One-to-many relationship

One reference row can be used by many detail records.

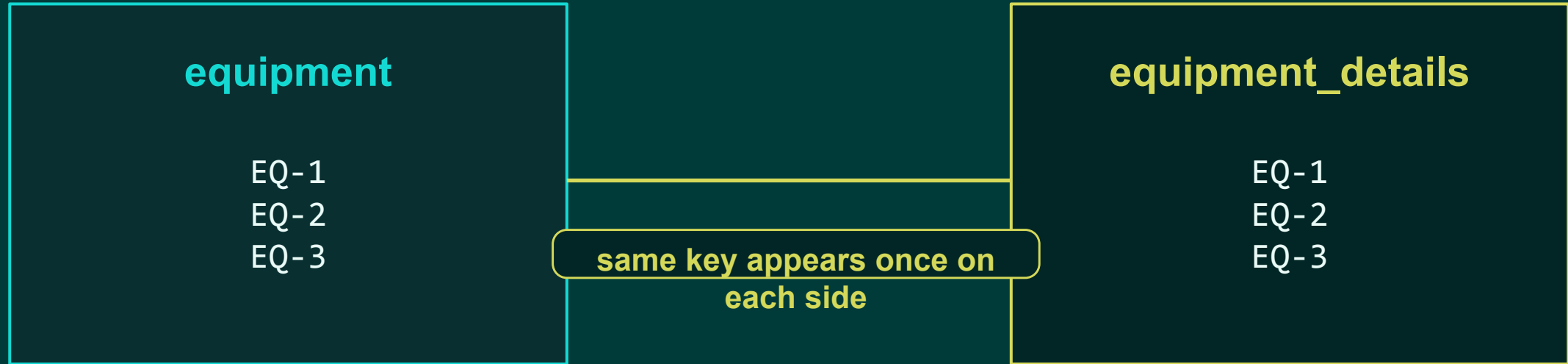


Read it as

one source
many
equipment
records

One-to-one relationship

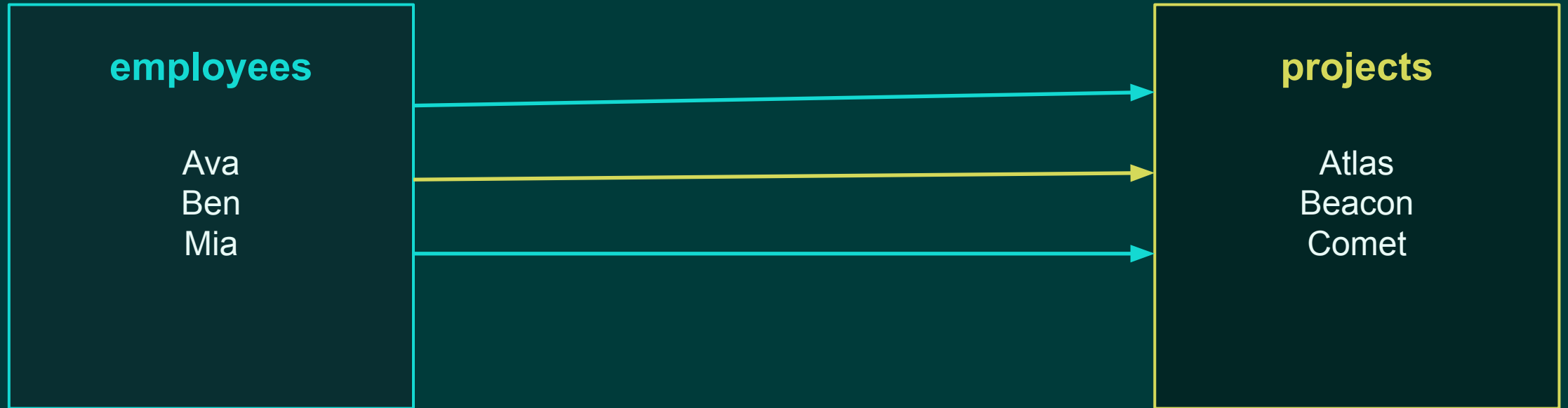
Each identifier appears once in both related tables.



Less common than one-to-many, but useful to recognize.

Many-to-many awareness

Some relationships need an intermediate table later.



Recognize the pattern now;
implement bridge tables later.

Missing references are information

An unmatched foreign key should be investigated, not automatically deleted.

record_id	source_code
EQ-1	MAINT
EQ-2	LOG
EQ-3	UNKNOWN

source_code	source_name
MAINT	Maintenance
LOG	Logistics
FIN	Finance

UNKNOWN has no match

Possibilities: typo, stale lookup table, new source, or bad source data.

Unmatched rows can reveal important data-quality problems.

Find unrecognized foreign keys before merge()

Compare the codes you have with the codes you recognize.

```
known_sources = set(
    source_reference["source_code"]
)

used_sources = set(
    equipment_records["source_code"]
)

unknown_sources = used_sources -
known_sources

print(unknown_sources)
```

Mental model

Which source codes appear in equipment records but do not appear in the reference table?

This uses the set difference idea from Day 3.

Key validation checklist

Run these checks before combining tables.

Missing keys

Are key columns blank or null?

Duplicate keys

Is the reference key unique?

Unknown references

Do detail rows point to missing lookup rows?

Relationship type

Is this one-to-one or one-to-many?

Joining before validating keys is how clean-looking tables become wrong tables.

Guided exercise setup

Use equipment_records.csv and source_reference.csv.

equipment_records

record_id	source_code	status	
errors			
EQ-1001	MAINT	active	0
EQ-1002	LOG	active	2
EQ-1003		inactive	0
EQ-1004	UNKNOWN	active	1

source_reference

source_code	source_name	region
MAINT	Maintenance	East
LOG	Logistics	West
LOG	Logistics Ops	West

The data intentionally contains problems.

Exercise 1: identify candidate keys

Goal: name the columns before writing validation code.

Questions

1. Which column identifies equipment rows?
2. Which column identifies source-reference rows?
3. Which column connects the tables?

Write this first

Do not start with merge().
Start with the relationship.

Timebox: 5 minutes

Exercise 2: check reference-key uniqueness

Goal: detect duplicate lookup keys.

```
print(
    source_reference["source_code"]
    .is_unique
)

source_duplicates = source_reference[
    source_reference.duplicated(
        subset=["source_code"],
        keep=False
    )
]

print(source_duplicates)
```

Task

Run the check. Explain why duplicate source codes matter before a join.

Timebox: 5 minutes

Exercise 3: check missing key values

Goal: count blank or missing values in key columns.

```
missing_equipment_sources = (  
    equipment_records["source_code"]  
    .isna()  
    .sum()  
)  
  
missing_reference_sources = (  
    source_reference["source_code"]  
    .isna()  
    .sum()  
)  
  
print(missing_equipment_sources)  
print(missing_reference_sources)
```

Note

If blanks were read as empty strings, normalize them to missing values or check for empty strings too.

Timebox: 5 minutes

BLOCK 46

Exercise 4: find unrecognized foreign keys

Goal: identify codes that appear in detail rows but not in reference rows.

```
known_sources = set(
    source_reference["source_code"].dropna()
)

used_sources = set(
    equipment_records["source_code"].dropna()
)

unknown_sources = used_sources - known_sources

print(unknown_sources)
```

Task

Record any unknown codes and decide whether they look like typos, new codes, or missing reference rows.

Timebox: 5 minutes

Exercise 5: draw the relationship

Goal: describe the relationship without code.



Label the primary key, the foreign key, and the expected relationship type.

Timebox: 5 minutes

Applied challenge: inspect three tables

No join yet. Reason about relationships first.

Tables

equipment_records
source_reference
organization_reference

Identify how these tables connect before writing merge code.

Required evidence

For each table, identify:

- candidate primary key
- foreign keys
- expected relationship type
- duplicate-key problems
- missing references

Success: explain the data model in plain English.

Vocabulary checkpoint

Use precise words to describe relationships.

Primary key

A column that identifies one row in its own table.

Foreign key

A column that points to a related row in another table.

Reference data

A lookup table that explains shared codes or categories.

One-to-many

One reference row relates to many detail rows.

Duplicate key

A supposed key appears more than once.

Missing reference

A foreign key points to no known reference row.

Block 46 wrap-up

You can now reason about connected tables before combining them.

Split

Separate detail facts from reusable reference descriptions.

Identify

Find primary keys, foreign keys, and relationship types.

Validate

Check uniqueness, missing keys, and unknown references.

Before joining tables, know what one row represents on each side and what the key is supposed to mean.

Next: use `pandas merge()` to combine related tables deliberately.